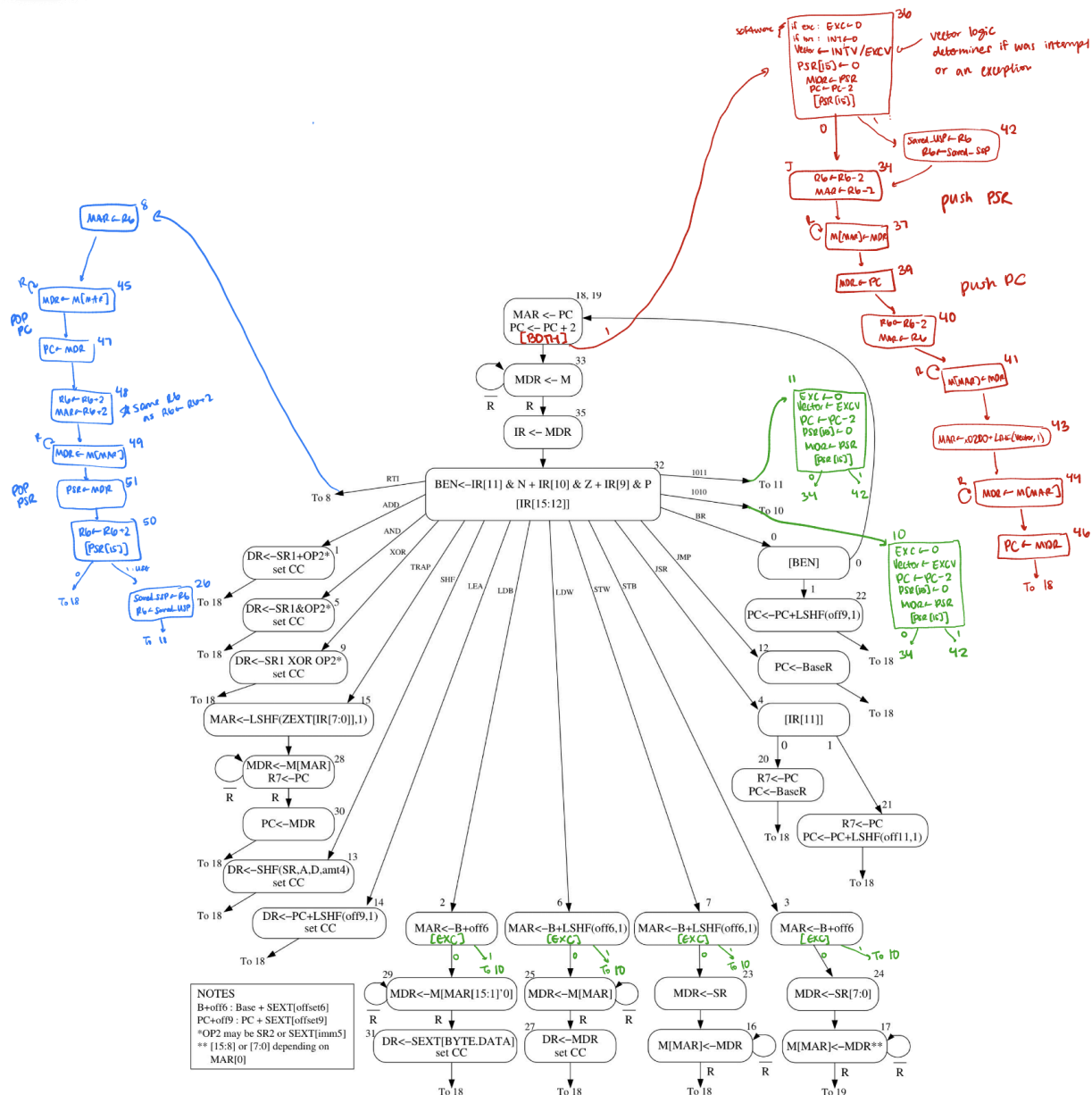
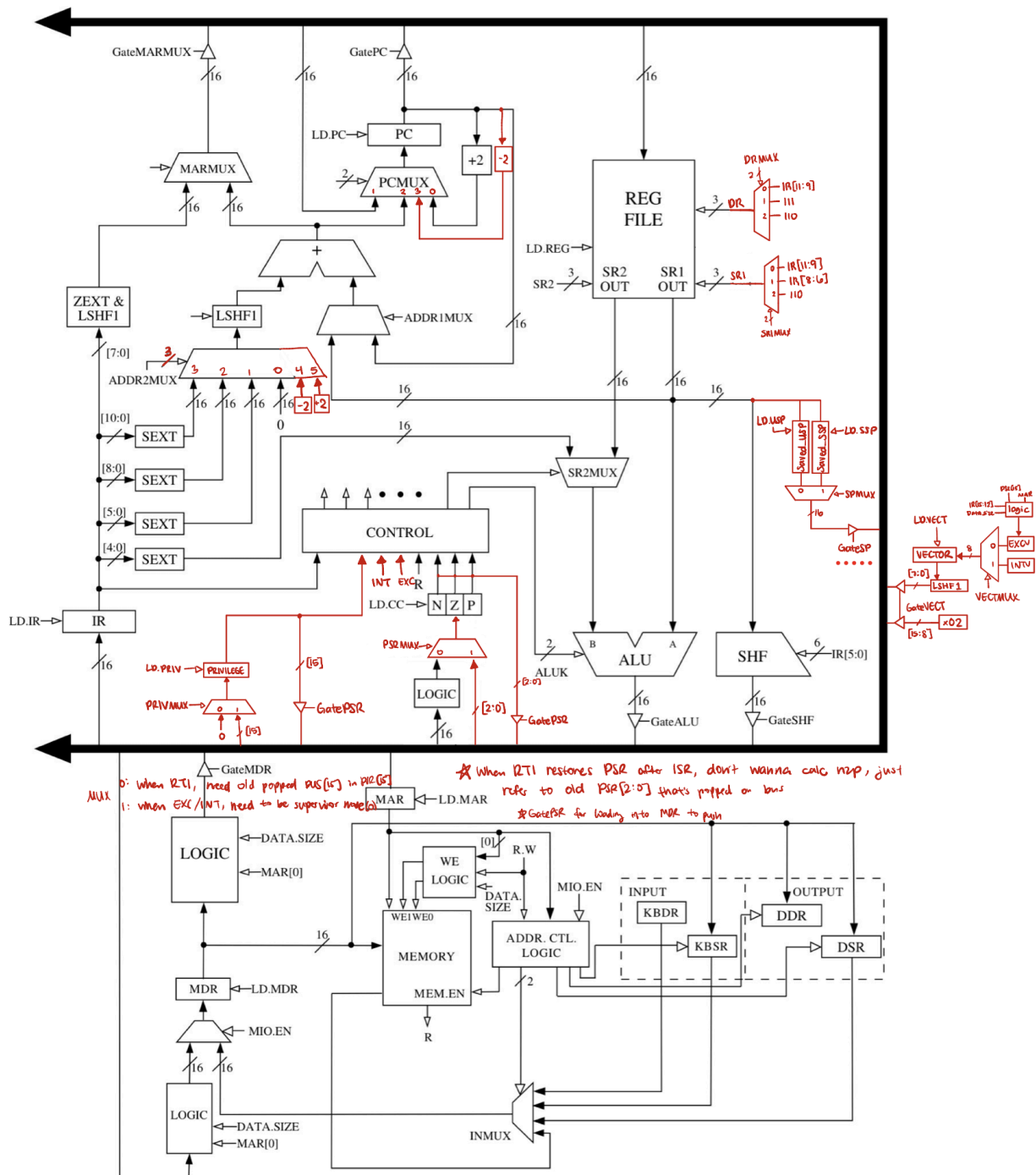


These are the changes I made to the state diagram:



In states 18/19, I check for both interrupts and exceptions. If this flag is high, we go to state 36. This is where it executes a set of states that loads the vector register with its corresponding signal, enters supervisor mode, saves R6, pushes PSR and PC onto the supervisor stack, and gets the vector's service routine's starting address (states 43, 44, 46). Similarly in states 2, 6, 7, and 3, where we are accessing memory, I check for exceptions. I made state 10 the default exception state that loads the vector with one of the 3 exception vectors and joins the series of states after 36. For RTI, I pop off the saved PC and PSR, then restore R6 if going back to the user program.

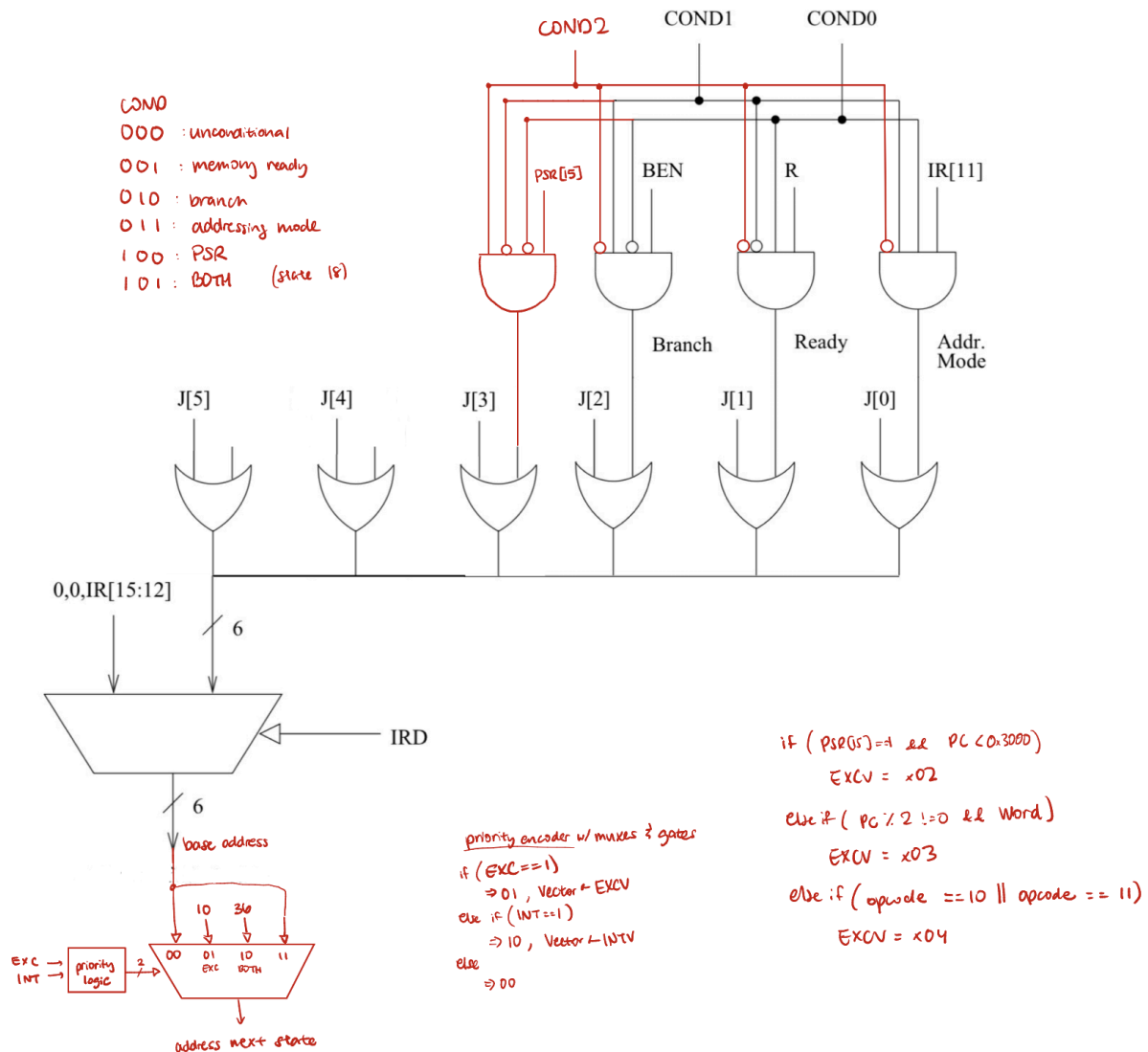
Here are changes I made to the datapath:



I made the empty 11_2 in PCMUX be PC-2. I added a bit to both DRMUX and SR1MUX so it can select R6. I also expanded ADDR2MUX to allow for R6-2 and R6+2. Then, I added a SPMUX to choose from the saved SSP or USP and wired the REG FILE to the inputs of both Saved_SSP and Saved_USP so I don't need the bus to update them with R6. I have a register to hold the privilege bit of the PSR, which either is cleared to 0, or gets restored to its original bit in RTI with the PRIVMUX. I have a similar PSRMUX that either sets condition codes with the original logic of looking at the value on the bus or restoring the original NZP from PSR[2:0]. Finally, I calculate the memory location which contains the starting address of interrupt/exception service routines. EXCV gets set to either x02, x03, or x04 with a logic block that evaluates the opcode, privilege bit, MAR, and DATA.SIZE to determine what exception is caused. VECTMUX selects either INTV (x01) or EXCV, and this value gets left shifted once, then added to 0x0200 to form the address. New control signals I added include:

- Extending COND to 3 bits, DRMUX to 2 bits, SR1MUX to 2 bits, ADDR2MUX to 3 bits to account for checking PSR[15] and selecting R6.
- LD.USP, LD.SSP, LD.VECT, LD.PRIV to load the registers holding this data
- GateSP, GateVECT, GatePSR to drive the bus with corresponding stack pointers, memory locations, and important PSR bits.
- SPMUX, VECTMUX, PSRMUX, PRIVMUX: select which input to choose based the conditions of what state is being executed

The next page show changes I made to my microsequencer:



I added an AND gate that evaluates if PSR[15] is a 1. If it is, it gets ORed with J[3] to get to the next state number that does the context switch between user and supervisor mode. My exceptions and interrupts get handled by a mux I added that chooses from either the originally calculated J bits, state 10 (exception), or state 36 (interrupt). Since exceptions have highest priority, if the exception flag is high, it will always go to state 10. Otherwise, if the interrupt flag is high, it'll go to state 36. If both flags are low, it goes to the J bits.